

NLP: Assignment 2 GS-2

Opened: Saturday, 1 August 2026, 12:00 AM

Due: Sunday, 23 August 2026, 11:59 PM

Work Integrated Learning Programmes Division

M.Tech. in AIML

Natural Language Processing

S2-25_AIMLCZG530

Assignment - 2 GS-2

General Instructions

1. Implementation on the BITS OSHA Virtual Lab is compulsory, and carries 1 mark.
2. Submit the code in .pdf/html format along with output. Please follow the naming convention as **<Group no>.pdf**. Example for group 1 should be named as - Group1.pdf.
3. Include group details like team members' names, BITS ID, and contribution percentages.
4. All members of the group will work on the same problem statement. Each group should upload in Taxila in respective locations under ASSIGNMENT Tab. Assignments submitted via means other than through Taxila will not be graded.
5. One student per group is requested to submit your assignment.
6. Any queries related to this problem statement should be addressed to Gourav Sharma, Course LF.
7. No dataset link is provided for this problem statement. Each group is free to choose any easily available public dataset that matches the dataset description given under Task 2. Clearly mention the dataset name, source, licence and size in your submission.
8. **Screenshots of the working application are compulsory.** The report must contain clear, readable screenshots of the running web application - the home/input screen, a sample input being submitted, and the generated output as displayed by the application interface. Reports submitted without application screenshots will lose marks under Task 4 and Task 6.
9. **All code files must be submitted together as a single ZIP archive**, named **<Group no>_Code.zip** (example for group 1 - **Group1_Code.zip**). The ZIP must contain:
 - all source code files and notebooks (training, inference and the Streamlit/Flask application),
 - **requirements.txt** listing every dependency with versions,
 - the saved model / tokenizer / vocabulary files, or a small script to download or regenerate them if the files are too large,
 - one or two sample input files for testing,
 - a **README.md** with **step-by-step local setup instructions**: Python version, virtual environment creation, dependency installation (**pip install -r requirements.txt**), where to place the dataset and model files, and the exact command to launch the application (for example **streamlit run app.py** or **flask run**), along with the local URL and any known issues. The evaluator should be able to set up and run your application on a local machine by following the README alone.

Automatic News Headline Generation

Problem Statement

News desks and content aggregation platforms publish hundreds of articles every day, and each one needs a short, accurate and attention-holding headline. Writing headlines manually is repetitive and does not scale, while poorly written headlines reduce readership and hurt search visibility. Develop an end-to-end Headline Generation Application that reads the body of a news article and automatically produces a single-line abstractive headline. The application should process article text and generate the headline using an Encoder–Decoder architecture.

Tasks to perform

1. Problem Analysis (2 Marks)

1. Identify the application domain and the stakeholders who would use such a system.
2. Define the problem statement and functional requirements, and explain how headline generation differs from extractive summarization.
3. Specify the expected input and output of the application, including the acceptable length range of the generated headline.

2. Data Collection and Preprocessing (2 Marks)

1. Select a suitable news dataset or collect sample articles.

Dataset description (choose any dataset that matches the following):

- A **news corpus** in which every record contains an **article body (or a descriptive paragraph)** and its corresponding **headline / title**.
 - Approximately **20,000 to 2,00,000 records**; a subset may be sampled to keep training time reasonable.
 - Article text of roughly **50–400 words** and headlines of roughly **5–15 words**.
 - Optional additional fields such as category, author or publication date may be used for filtering or analysis.
 - Such news-with-headline datasets are widely available on open dataset repositories, community model hubs, news-archive dumps and public research corpora; groups may also scrape openly licensed RSS feeds.
2. Perform text preprocessing, including tokenization, cleaning of HTML/boilerplate text, lowercasing, normalization, removal of very short or empty records and addition of start/end-of-sequence tokens.
 3. Prepare the dataset for model training and testing by building vocabularies, truncating articles to a fixed input length, padding sequences and creating train–validation–test splits.

3. Model Development (3 Marks)

1. Build an Encoder–Decoder (Seq2Seq/LSTM or Transformer) model; use of an attention mechanism between encoder and decoder is encouraged.
2. Train the model to generate abstractive headlines, reporting the loss curves and the training configuration used.
3. Generate headlines for the given input articles using greedy or beam-search decoding.

4. Application Development (3 Marks)

1. Develop a web application using Streamlit or Flask.
2. Provide options for text input or document upload (.txt/.csv) so that multiple articles can be processed.
3. Display the generated headline through the application interface, optionally showing the top-k candidate headlines.

5. Evaluation and Demonstration (2 Marks)

1. Assess the generated headlines using ROUGE or other suitable evaluation metrics, and report scores on the held-out test set.
2. Demonstrate the application using sample inputs and outputs from at least two different news categories, and include the corresponding application screenshots in the report.
3. Present observations based on the generated headlines, covering factual consistency, repetition, generic wording and handling of named entities.

6. Project Report (1 Mark)

1. Include implementation details, **screenshots of the working application** (input screen, sample input submitted, and the generated output shown by the application), observations, conclusion, and references in a well-structured report. (include as a pdf)
2. Attach the complete code as a single ZIP archive along with a **README.md** containing the local setup and execution instructions, as described in General Instruction 9.